

Lesson 3: Decisions

Faculty of Data Science & Artificial Intelligence (FDA) - National Economics University

Course Code: DSAI1003

Contents

Lesson 3. Decisions	1
Lesson Introduction	1
Knowledge and Skills to Be Achieved	2
Lesson Structure	2
3.1 The if Statement	2
3.1.1 A Two-Way Decision	2
3.1.2 General Syntax	3
3.1.3 Compound Statements and Blocks	3
Example – Elevator Simulation	4
Common Error 3.1 – Tabs and Inconsistent Indentation	4
Programming Tip 3.1 – Avoid Duplication in Branches	4
Special Topic 3.1 – Conditional Expressions	5
Self Check 3.1	5
Question 1	5
Question 2	5
Question 3	5
Question 4	6
3.2 Relational Operators	6
3.2.1 Relational Operators	6
3.2.2 Assignment = versus Equality ==	6
3.2.3 Comparing Strings	7
3.2.4 Operator Precedence	7
Common Error 3.2 – Exact Comparison of Floating-Point Numbers	7
Special Topic 3.2 – Lexicographic Ordering of Strings	8
HOW TO 3.1 – Implementing an if Statement	8
Step 1. Decide on the branching condition	8
Step 2. Describe what happens when the condition is true	8
Step 3. Describe what happens when the condition is false	9
Step 4. Check the relational operator carefully	9
Step 5. Remove duplicated work	9
Step 6. Test both branches	9
Step 7. Assemble the Python program	9
Worked Example 3.1 – Extracting the Middle of a String	10

Self Check 3.2	10
Question 1	10
Question 2	10
Question 3	11
Question 4	11
3.3 Nested Branches	11
3.3.1 Multi-Level Decisions	11
Programming Tip 3.2 – Hand-Tracing	12
Computing & Society 3.1 – Complex Systems and Decision Logic	12
Self Check 3.3	13
Question 1	13
Question 2	13
Question 3	13
3.4 Multiple Alternatives	13
Example – Classifying an Earthquake	13
3.4.1 Order Matters	14
3.4.2 elif versus Independent if Statements	14
Self Check 3.4	15
Question 1	15
Question 2	15
TOOLBOX 3.1 – Sending E-mail	15
3.5 Problem Solving: Flowcharts	16
3.5.1 Two-Way Decision	16
3.5.2 Multiple Alternatives	16
3.5.3 Avoiding Spaghetti Code	16
Example – Shipping Cost	16
Self Check 3.5	17
Question 1	17
Question 2	17
3.6 Problem Solving: Test Cases	17
3.6.1 Branch Coverage	17
3.6.2 Boundary Cases	18
3.6.3 Design Tests Before Coding	18
Programming Tip 3.3 – Plan Time for Testing and Debugging	18
Self Check 3.6	19
Question 1	19
Question 2	19
3.7 Boolean Variables and Operators	19
3.7.1 The and Operator	19
3.7.2 The or Operator	20
3.7.3 The not Operator	20
3.7.4 Precedence	20
Common Error 3.3 – Confusing and and or	21
Programming Tip 3.4 – Readability	21

Special Topic 3.3 – Chaining Relational Operators	21
Special Topic 3.4 – Short-Circuit Evaluation	22
Special Topic 3.5 – De Morgan’s Laws	22
Self Check 3.7	23
Question 1	23
Question 2	23
Question 3	23
Question 4	23
3.8 Analyzing Strings	23
3.8.1 Membership with in and not in	23
3.8.2 Useful Substring Methods	24
3.8.3 Testing String Characteristics	24
Example – Exploring a Substring	25
Self Check 3.8	25
Question 1	25
Question 2	25
Question 3	25
3.9 Application: Input Validation	25
3.9.1 Range Validation	26
3.9.2 Validating Choice Inputs	26
3.9.3 A Limitation at This Stage	27
Special Topic 3.6 – Terminating a Program	27
Special Topic 3.7 – Interactive Graphical Programs	27
Computing & Society 3.2 – Artificial Intelligence	28
Worked Example 3.2 – Intersecting Circles	28
TOOLBOX 3.2 – Plotting Simple Graphs	29
Chapter 3 Summary	29
if Statements	29
Relational Operators	29
Nested Branches	30
Multiple Alternatives	30
Flowcharts	30
Test Cases	30
Boolean Logic	30
String Analysis	31
Input Validation	31
Summary Quiz	31
Question 1	31
Question 2	31
Question 3	32
Question 4	32
Question 5	32
Question 6	32
Question 7	33
Question 8	33

Question 9	33
Question 10	33
Question 11	33
Question 12	33
Review Exercises	34
Exercise 1. Trace a Two-Way Decision	34
Exercise 2. Choose the Correct Relational Operator	34
Exercise 3. Rewrite Without Duplication	34
Exercise 4. Test Cases	35
Exercise 5. Boolean Expressions	35
Exercise 6. Analyze a String	35
Practice Exercises	35
Exercise 1. Pass / Fail / Invalid Score	35
Exercise 2. Largest of Two Numbers	36
Exercise 3. Largest of Three Numbers	36
Exercise 4. Shipping Rule	36
Exercise 5. Grade Classification	36
Exercise 6. Username Validation	36
Exercise 7. File-Type Classifier	37
Exercise 8. Approximate Equality	37
Exercise 9. Triangle Type from Side Lengths	37
Exercise 10. Simple Login-Code Check	37
Open Exercises	38
Exercise 1	38
Exercise 2	38
Exercise 3	38
Exercise 4	38
Exercise 5	38
Key Terms	38
Study Guidance	39

Lesson 3. Decisions

Last updated: September 19, 2026

Source: Chapter 3 – *Decisions*, in the uploaded *Python for Everyone* textbook by Cay Horstmann and Rance Necaise (2/e).

This lecture follows the structure, terminology, examples, and instructional focus of Chapter 3 while reorganizing the material for direct classroom use. The examples use the textbook's naming conventions and, where formatting is needed, its %-style string formatting.

Lesson Introduction

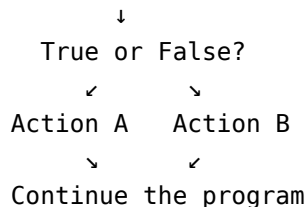
The programs in Lesson 2 executed statements mostly from top to bottom. That is enough for straightforward calculations, but useful programs must also be able to **make decisions**.

A decision allows a program to choose different actions depending on data or circumstances. For example, a program may need to:

- apply one discount for small purchases and another discount for large purchases;
- reject an invalid floor number in an elevator program;
- classify an earthquake by magnitude;
- determine whether a string has the required form;
- choose a tax calculation according to marital status and income;
- check whether several conditions are simultaneously true.

The central idea of this lesson is:

Evaluate a condition



The chapter develops this idea gradually, beginning with a two-way `if` statement and then extending it to nested decisions, multiple alternatives, Boolean expressions, string analysis, systematic testing, and input validation.

Knowledge and Skills to Be Achieved

After completing this lesson, students will be able to:

- Use `if`, `if/else`, and `if/elif/else` statements.
 - Explain the role of a **condition** in program control flow.
 - Use indentation correctly to define statement blocks.
 - Compare numeric and string values using relational operators.
 - Distinguish assignment `=` from equality testing `==`.
 - Avoid exact equality tests for computed floating-point values when approximation is required.
 - Design a two-way decision systematically from a problem statement.
 - Implement **nested branches** for multi-level decisions.
 - Use `elif` for mutually exclusive alternatives.
 - Order conditions correctly from more specific to more general cases when necessary.
 - Draw and interpret simple **flowcharts**.
 - Design **test cases** that cover branches and boundary values.
 - Use Boolean values `True` and `False`.
 - Combine conditions using `and`, `or`, and `not`.
 - Explain operator precedence and short-circuit evaluation.
 - Apply De Morgan's laws to simplify negated Boolean expressions.
 - Analyze strings using membership operators and string methods.
 - Validate user input before processing it.
 - Normalize text input with `upper()` or `lower()` when case should not matter.
-

Lesson Structure

1. The `if` Statement
2. Relational Operators
3. Nested Branches
4. Multiple Alternatives
5. Problem Solving: Flowcharts
6. Problem Solving: Test Cases
7. Boolean Variables and Operators

- 8. Analyzing Strings
 - 9. Application: Input Validation
 - 10. Optional Topics and Toolboxes
 - 11. Summary and Exercises
-

3.1 The if Statement

Programs often need to perform one action when a condition is true and another action when it is false.

The if statement provides this branching behavior.

3.1.1 A Two-Way Decision

Consider an elevator system in a building that labels the floor after 12 as floor 14. Internally, the physical floor numbers still continue normally, so a request for displayed floor 20 corresponds to actual floor 19.

A decision can be written as:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor
```

Conceptually:

```

      floor > 13 ?
      /         \
    True        False
    /           \
actualFloor = floor - 1  actualFloor = floor
```

Only **one branch** executes.

If `floor > 13` is true, Python executes the first block and skips the `else` block. Otherwise, Python skips the first block and executes the `else` block.

3.1.2 General Syntax

```
if condition :
    statements1
else :
    statements2
```

The `else` branch is optional:

```
if condition :
    statements
```

Example:

```
actualFloor = floor
```

```
if floor > 13 :
    actualFloor = actualFloor - 1
```

This version is useful when no special action is required for the false case.

3.1.3 Compound Statements and Blocks

An if statement is a **compound statement**. Its header ends with a colon, and the statements belonging to the branch are indented.

```
if totalSales > 100.0 :  
    discount = totalSales * 0.05  
    totalSales = totalSales - discount  
    print("You received a discount of", discount)
```

The three indented statements form one block.

Important rules:

- The header ends with :.
- Statements in the same block must be indented to the same level.
- The if and its matching else must be aligned.
- The block ends when the indentation returns to an outer level.

Python uses indentation as part of its syntax, not merely as decoration.

Example – Elevator Simulation

```
##  
# This program simulates an elevator panel that skips floor 13.  
#  
  
floor = int(input("Floor: "))  
  
if floor > 13 :  
    actualFloor = floor - 1  
else :  
    actualFloor = floor  
  
print("The elevator will travel to the actual floor", actualFloor)
```

Example run:

```
Floor: 20  
The elevator will travel to the actual floor 19
```

Common Error 3.1 – Tabs and Inconsistent Indentation

Python requires statements in one block to have consistent indentation.

Problematic code can look aligned in an editor but internally mix tab characters and spaces.

Prefer configuring the editor to insert spaces when the Tab key is pressed.

A common convention is four spaces per indentation level:

```
if score >= 50 :  
    print("Pass")  
    print("Continue to the next course")
```

The important point is consistency.

Programming Tip 3.1 – Avoid Duplication in Branches

Consider:

```
if floor > 13 :
    actualFloor = floor - 1
    print("Actual floor:", actualFloor)
else :
    actualFloor = floor
    print("Actual floor:", actualFloor)
```

The output statement is duplicated.

A clearer version is:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor
```

```
print("Actual floor:", actualFloor)
```

Moving common work outside the branches:

- reduces duplication;
 - makes the program shorter;
 - reduces the risk that one copy is changed while another is forgotten.
-

Special Topic 3.1 – Conditional Expressions

Python can represent a simple two-way choice as an expression:

```
actualFloor = floor - 1 if floor > 13 else floor
```

This is equivalent to:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor
```

For beginning programs, the full if/else form is often easier to read, especially when each branch contains more than one action.

Self Check 3.1

Question 1

What value is stored in actualFloor when floor is 18?

```
if floor > 13 :
    actualFloor = floor - 1
```

```
else :  
    actualFloor = floor
```

Answer

17.

Question 2

What value is stored in actualFloor when floor is 9?

Answer

9.

Question 3

Why is the colon required after the if condition?

Answer

It marks the header of a compound statement. The following indented block belongs to that header.

Question 4

How can this code be improved?

```
if balance < 0 :  
    status = "Overdrawn"  
    print(status)  
else :  
    status = "OK"  
    print(status)
```

Answer

Move the duplicated print statement outside the branches.

```
if balance < 0 :  
    status = "Overdrawn"  
else :  
    status = "OK"  
  
print(status)
```

3.2 Relational Operators

An if statement needs a condition whose value is either True or False.

Many conditions compare two values using a **relational operator**.

3.2.1 Relational Operators

Python operator	Meaning
>	Greater than
>=	Greater than or equal to
<	Less than

Python operator	Meaning
<=	Less than or equal to
==	Equal to
!=	Not equal to

Examples:

```
3 < 4
4 <= 4
7 != 5
10 == 2 * 5
```

Each expression evaluates to a Boolean value.

3.2.2 Assignment = versus Equality ==

These operators have very different purposes.

Assignment:

```
floor = 13
```

Equality test:

```
if floor == 13 :
    print("Floor 13 selected")
```

A useful reading rule is:

```
=    → assign
==   → is equal to?
```

3.2.3 Comparing Strings

Strings can also be compared.

```
name1 = "John Wayne"
name2 = "John Wayne"
```

```
if name1 == name2 :
    print("The strings are identical.")
```

String comparison is case-sensitive:

```
"John" == "john"
```

is False.

For equality, every character and its position must match.

3.2.4 Operator Precedence

Arithmetic operators have higher precedence than relational operators.

```
floor - 1 < 13
```

Python first computes:

```
floor - 1
```

and then compares the result with 13.

For complicated expressions, parentheses can still improve readability.

Common Error 3.2 – Exact Comparison of Floating-Point Numbers

Floating-point calculations may contain tiny roundoff differences.

For example:

```
from math import sqrt
```

```
x = sqrt(2.0)
```

```
print(x * x)
```

The result may be extremely close to 2.0 without being represented internally as exactly 2.0.

Therefore, this test may fail:

```
if x * x == 2.0 :  
    print("Exactly two")
```

When approximate equality is intended, compare the difference with a small tolerance:

```
EPSILON = 1E-14
```

```
if abs(x * x - 2.0) < EPSILON :  
    print("Approximately two")
```

General pattern:

```
if abs(x - y) < epsilon :  
    # Treat x and y as approximately equal.
```

The tolerance should be chosen according to the scale and purpose of the computation.

Special Topic 3.2 – Lexicographic Ordering of Strings

Relational operators such as < and > can be used with strings.

Python compares strings lexicographically according to character codes.

For example, capitalization matters:

```
"John" < "john"
```

The result depends on the character ordering, not on dictionary rules that ignore case.

For case-insensitive comparisons, a useful technique is to normalize both strings first:

```
name1 = name1.lower()
```

```
name2 = name2.lower()
```

and then compare them.

HOW TO 3.1 – Implementing an if Statement

The chapter proposes a systematic approach to decisions.

Suppose a store applies one discount rate to prices below a threshold and a larger discount at or above the threshold.

Step 1. Decide on the branching condition

Ask a question that can be answered True or False.

Example:

Is the original price less than 128?

Step 2. Describe what happens when the condition is true

Use the lower discount rate.

Step 3. Describe what happens when the condition is false

Use the higher discount rate.

Step 4. Check the relational operator carefully

Ask what should happen at the boundary.

If the higher discount begins at exactly 128, then the lower-rate condition should be:

```
originalPrice < 128
```

not:

```
originalPrice <= 128
```

Boundary values often reveal incorrect < versus <= choices.

Step 5. Remove duplicated work

Instead of computing the entire discounted price in each branch, choose only the rate inside the branches:

```
if originalPrice < 128 :  
    discountRate = 0.92  
else :  
    discountRate = 0.84
```

```
discountedPrice = discountRate * originalPrice
```

Step 6. Test both branches

Choose at least one value below the threshold and one above it.

For example:

100 → lower-discount branch
200 → higher-discount branch
128 → boundary case

Step 7. Assemble the Python program

```
originalPrice = float(input("Original price before discount: "))
```

```
if originalPrice < 128 :  
    discountRate = 0.92  
else :  
    discountRate = 0.84
```

```
discountedPrice = discountRate * originalPrice  
print("Discounted price: %.2f" % discountedPrice)
```

The important skill is not memorizing this particular price example. It is learning how to translate a two-case rule into a correct condition and two branches.

Worked Example 3.1 – Extracting the Middle of a String

Suppose a program must extract:

- one middle character from an odd-length string;
- two middle characters from an even-length string.

Examples:

"crate" → "a"
"crates" → "at"

The branching condition is based on parity:

```
len(text) % 2 == 1
```

The middle position can be computed once:

```
position = len(text) // 2
```

Then:

```
if len(text) % 2 == 1 :  
    result = text[position]  
else :  
    result = text[position - 1] + text[position]
```

This example combines:

- string length;
- floor division;
- remainder;
- indexing;

- a two-way decision;
 - removal of duplicated calculations.
-

Self Check 3.2

Question 1

What is the value of:

```
4 <= 4
```

Answer

True.

Question 2

Why is this incorrect?

```
if score = 100 :  
    print("Perfect")
```

Answer

= performs assignment. Equality testing requires ==.

Question 3

What condition tests whether n is odd?

Answer

```
n % 2 == 1
```

Question 4

What is the middle result for:

```
text = "monitor"
```

Answer

"i".

3.3 Nested Branches

Sometimes a program must make one decision and then make another decision inside the selected branch.

This is called **nesting**.

Example structure:

```
if condition1 :  
    if condition2 :  
        statementA  
    else :  
        statementB  
else :  
    statementC
```

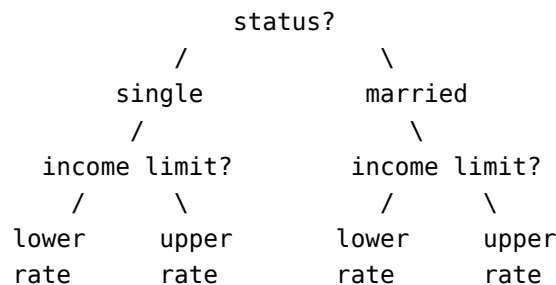
The second `if` is part of the first branch.

3.3.1 Multi-Level Decisions

A useful example is a simplified tax calculation in which:

1. the program first determines a taxpayer category;
2. it then determines which income bracket applies within that category.

Conceptually:



The important point is the structure: **one decision depends on the result of another decision.**

A simplified implementation pattern is:

```

if status == "s" :
    if income <= SINGLE_LIMIT :
        # lower bracket for single
        ...
    else :
        # upper bracket for single
        ...
else :
    if income <= MARRIED_LIMIT :
        # lower bracket for married
        ...
    else :
        # upper bracket for married
        ...

```

Nested decisions can go deeper, but excessive nesting makes programs harder to read. Later lessons will introduce functions as one way to organize more complex logic.

Programming Tip 3.2 – Hand-Tracing

Hand-tracing means simulating the program manually, one statement at a time.

A useful method is to create a small table:

Step	income	status	tax1	tax2
Input	80000	"m"		
Initialize	80000	"m"	0	0
Branch	80000	"m"	...	...

As each assignment occurs, update the affected variable.

Hand-tracing is especially useful for:

- nested `if` statements;
 - code with several variables;
 - understanding why a program entered a particular branch;
 - comparing expected and actual program behavior.
-

Computing & Society 3.1 – Complex Systems and Decision Logic

The chapter uses the Denver International Airport baggage-handling system as a case study in software and system complexity. The broader lesson is that decision logic that appears manageable in isolation can become difficult when many physical components, timing constraints, exceptional cases, and interactions are combined.

For beginning programmers, the practical lesson is:

Keep each decision understandable, test the branches systematically, and do not assume that a large system is simply a collection of independently correct small pieces.

Self Check 3.3

Question 1

When is nesting appropriate?

Answer

When a second decision only makes sense after a particular result of an earlier decision.

Question 2

Why is indentation particularly important in nested decisions?

Answer

Indentation determines which outer branch contains each inner statement.

Question 3

What does hand-tracing help you understand?

Answer

The order of execution, which branches are selected, and how variable values change.

3.4 Multiple Alternatives

Many decisions have more than two possible outcomes.

Python uses `elif` to express a sequence of mutually exclusive alternatives.

General form:

```
if condition1 :  
    statements1  
elif condition2 :
```

```
    statements2
elif condition3 :
    statements3
else :
    defaultStatements
```

Python evaluates the conditions from top to bottom. As soon as one condition is true, its branch executes and the remaining alternatives are skipped.

Example – Classifying an Earthquake

A simplified classification may use thresholds such as:

```
richter = float(input("Enter a magnitude: "))

if richter >= 8.0 :
    description = "Very severe structural damage"
elif richter >= 7.0 :
    description = "Severe damage in many areas"
elif richter >= 6.0 :
    description = "Considerable damage is possible"
elif richter >= 4.5 :
    description = "Some poorly constructed buildings may be damaged"
else :
    description = "Little or no structural damage"

print(description)
```

The exact descriptive labels are less important than the branching structure.

3.4.1 Order Matters

Suppose the conditions are written from the smallest threshold upward:

```
if richter >= 4.5 :
    ...
elif richter >= 6.0 :
    ...
elif richter >= 7.0 :
    ...
```

For `richter = 7.1`, the first condition is already true, so later branches are never considered.

When ranges overlap, test the **more specific / higher threshold** first.

Correct pattern:

```
if value >= 80 :
    ...
elif value >= 70 :
    ...
elif value >= 60 :
    ...
else :
    ...
```

3.4.2 elif versus Independent if Statements

These are not equivalent.

Mutually exclusive alternatives:

```
if score >= 90 :
    grade = "A"
elif score >= 80 :
    grade = "B"
elif score >= 70 :
    grade = "C"
```

Only one branch executes.

Independent tests:

```
if score >= 90 :
    print("At least 90")
if score >= 80 :
    print("At least 80")
if score >= 70 :
    print("At least 70")
```

For score = 95, all three statements print.

Use if/elif/else when alternatives are intended to be mutually exclusive. Use separate if statements when several conditions may independently trigger actions.

Self Check 3.4

Question 1

Write the logic for setting sign to 1, 0, or -1 depending on whether x is positive, zero, or negative.

Answer

```
if x > 0 :
    sign = 1
elif x < 0 :
    sign = -1
else :
    sign = 0
```

Question 2

Why should overlapping threshold conditions usually be ordered from largest to smallest?

Answer

Because the first true branch ends the if/elif chain. A broad low-threshold condition placed first can capture values that should belong to a later, more specific case.

TOOLBOX 3.1 – Sending E-mail

The textbook uses e-mail automation as an example of combining data, decisions, and a Python library.

The key programming idea is to construct different message content according to a condition:

```
score = int(input("Score: "))
body = "Your score on the last exam is " + str(score) + "\n"

if score <= 50 :
    body += "Please consider visiting the tutoring center."
elif score >= 90 :
    body += "Excellent work."
```

The library-specific details are optional for this lesson. The decision-making concept is that the program can customize an action or message according to data.

For real accounts, authentication and mail-provider requirements change over time. Do not place real passwords directly in teaching code or shared notebooks.

3.5 Problem Solving: Flowcharts

A **flowchart** visualizes the flow of control in an algorithm.

Common elements include:

[Process / task]

< Input / output >

```
      / Condition? \
     /              \
    True             False
```

Flowcharts help when a problem contains several decisions and it is not yet clear how the branches should fit together.

3.5.1 Two-Way Decision

```
      temperature < 0 ?
       /              \
      True             False
       |               |
    print("Frozen")   continue
```

3.5.2 Multiple Alternatives

```
      score >= 90 ?
       /          \
      True         False
       |           |
    grade A       score >= 80 ?
                   /          \
```

True	False
grade B	...

3.5.3 Avoiding Spaghetti Code

A useful design rule from the chapter is:

Do not draw an arrow from one branch into the middle of another branch.

Branches should be structured so that they clearly split and later rejoin.

This keeps the control flow consistent with structured `if`, `if/else`, and nested statements.

Example – Shipping Cost

Suppose:

- shipping inside the continental USA costs one rate;
- shipping to Alaska or Hawaii costs another rate;
- international shipping uses the higher rate.

A structured solution is:

```
country = input("Enter the country: ")
state = input("Enter the state or province: ")

if country == "USA" :
    if state == "AK" or state == "HI" :
        shippingCost = 10.0
    else :
        shippingCost = 5.0
else :
    shippingCost = 10.0

print("Shipping cost to %s, %s: $%.2f" %
      (state, country, shippingCost))
```

The same structure can first be drawn as a flowchart and then translated into Python.

Self Check 3.5

Question 1

What is the main purpose of a flowchart?

Answer

To visualize tasks, decisions, and the possible paths of execution before or while implementing the algorithm.

Question 2

Why should arrows not enter the middle of another branch?

Answer

It creates unstructured control flow that is difficult to translate, test, and maintain.

3.6 Problem Solving: Test Cases

A program with decisions must be tested with inputs that exercise the different paths through the program.

Trying one ordinary input is not enough.

A strong test plan aims for:

1. **branch coverage** – each branch is executed by at least one test;
 2. **boundary testing** – values exactly at or very near decision boundaries are tested;
 3. **invalid-input testing** – when the program is responsible for validating input.
-

3.6.1 Branch Coverage

For a two-way decision:

```
if age >= 18 :  
    category = "Adult"  
else :  
    category = "Minor"
```

Use at least one test for each outcome:

Input	Expected result	Purpose
20	Adult	True branch
15	Minor	False branch

3.6.2 Boundary Cases

The boundary is 18.

Useful tests include:

17
18
19

The value 18 is particularly important because it distinguishes `>=` from `>`.

3.6.3 Design Tests Before Coding

Writing expected results before implementation has several benefits:

- forces you to understand the rule;
- reveals ambiguous boundary conditions;
- gives you an immediate way to verify the finished program;
- reduces the temptation to accept whatever output the program happens to produce.

A useful test table is:

Test case	Expected output	Reason
ordinary case A	...	first branch
ordinary case B	...	second branch
boundary	...	exact cutoff
invalid value	error	validation

Programming Tip 3.3 – Plan Time for Testing and Debugging

Programming work includes more than typing code. A realistic plan includes time for:

- understanding and designing the solution;
- preparing test cases;
- entering the program and correcting syntax errors;
- running tests and debugging unexpected behavior.

Programs with more branches have more possible execution paths and therefore usually require more testing effort.

Self Check 3.6

Question 1

For the condition:

```
if value < 100 :
```

what is an important boundary test?

Answer

100. Values such as 99 and 101 are also useful neighbors.

Question 2

Why is one test case usually insufficient for an if/else statement?

Answer

It may execute only one branch, leaving the other branch untested.

3.7 Boolean Variables and Operators

The Boolean type `bool` has exactly two values:

```
True  
False
```

A Boolean variable can store the result of a logical condition.

```
failed = True
```

```
if failed :  
    print("The operation failed")
```

A Boolean variable is sometimes called a **flag** because its state indicates whether a condition holds.

3.7.1 The and Operator

and is true only when **both** operands are true.

```
if temp > 0 and temp < 100 :  
    print("Liquid water")
```

Truth table:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

3.7.2 The or Operator

or is true when **at least one** operand is true.

```
if temp <= 0 or temp >= 100 :  
    print("Not liquid water")
```

Truth table:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Notice that or is inclusive: it is still true when **both** conditions are true.

3.7.3 The not Operator

not reverses a Boolean value.

```
if not frozen :  
    print("Not frozen")
```

A	not A
True	False
False	True

3.7.4 Precedence

A useful simplified order is:

```
arithmetic
  ↓
relational operators
  ↓
not
  ↓
and
  ↓
or
```

Thus:

```
x > 0 and y < 10
```

is interpreted as:

```
(x > 0) and (y < 10)
```

For complex expressions, parentheses improve readability even when they are not required by precedence.

Common Error 3.3 – Confusing and and or

Suppose a value is valid only when it lies between 0 and 100.

Correct:

```
if value >= 0 and value <= 100 :
    print("Valid")
```

Both conditions must hold.

To detect an invalid value:

```
if value < 0 or value > 100 :
    print("Invalid")
```

Only one of the invalid conditions needs to hold.

A helpful question is:

- Must **all** conditions be true? → use and.
 - Is **any one** sufficient? → use or.
-

Programming Tip 3.4 – Readability

Avoid comparing Boolean variables explicitly with True or False.

Less readable:

```
if frozen == False :
    print("Not frozen")
```

Prefer:

```
if not frozen :  
    print("Not frozen")
```

Likewise:

```
if valid :  
    print("Input accepted")
```

is usually clearer than:

```
if valid == True :  
    print("Input accepted")
```

Boolean variable names should describe a condition clearly:

```
valid  
finished  
found  
isEmpty  
hasError
```

Special Topic 3.3 – Chaining Relational Operators

Python allows:

```
0 <= value <= 100
```

This is equivalent to:

```
value >= 0 and value <= 100
```

Chained comparisons are concise and Pythonic, but beginners should understand the equivalent Boolean expression explicitly.

Special Topic 3.4 – Short-Circuit Evaluation

Python evaluates and and or from left to right and stops as soon as the final result is known.

Example:

```
quantity > 0 and price / quantity < 10
```

If `quantity > 0` is false, Python does not evaluate the division. This prevents division by zero when `quantity` is 0.

For or:

```
conditionA or conditionB
```

if `conditionA` is already true, `conditionB` is not evaluated.

This behavior is called **short-circuit evaluation**.

Special Topic 3.5 – De Morgan's Laws

De Morgan's laws help rewrite negated compound conditions.

`not (A and B)   ≡  (not A) or (not B)`

`not (A or B)   ≡  (not A) and (not B)`

Example:

```
not (state == "AK" or state == "HI")
```

is equivalent to:

```
state != "AK" and state != "HI"
```

Another example:

```
not (country == "USA" and state != "AK" and state != "HI")
```

can be rewritten more directly as:

```
country != "USA" or state == "AK" or state == "HI"
```

The goal is not merely shorter code; it is clearer logic.

Self Check 3.7

Question 1

How do you test whether both x and y are positive?

Answer

```
x > 0 and y > 0
```

Question 2

How do you test whether at least one of x and y is zero?

Answer

```
x == 0 or y == 0
```

Question 3

What is the value of:

```
not not True
```

Answer

True.

Question 4

Why can this be safe when `quantity == 0`?

```
quantity > 0 and price / quantity < 10
```

Answer

Because short-circuit evaluation stops after the first false condition, so the division is not attempted.

3.8 Analyzing Strings

Decision logic is often used to examine text.

Python provides operators and methods for checking whether a string contains another string or has particular characteristics.

3.8.1 Membership with `in` and `not in`

```
name = "John Wayne"
```

```
if "Way" in name :  
    print("Substring found")
```

The inverse test is:

```
if "-" not in name :  
    print("The name does not contain a hyphen")
```

Membership tests are case-sensitive.

3.8.2 Useful Substring Methods

Operation	Meaning
substring in s	Does s contain substring?
s.count(substring)	Number of non-overlapping occurrences
s.startswith(substring)	Does s begin with it?
s.endswith(substring)	Does s end with it?
s.find(substring)	Index of first occurrence, or -1 if not found

Example:

```
filename = "report.html"
```

```
if filename.endswith(".html") :  
    print("HTML file")
```

Example:

```
text = "banana"  
print(text.count("an"))  
print(text.find("na"))
```

3.8.3 Testing String Characteristics

Useful methods include:

Method	True when...
<code>s.isalnum()</code>	all characters are letters or digits and the string is nonempty
<code>s.isalpha()</code>	all characters are letters and the string is nonempty
<code>s.isdigit()</code>	all characters are digits and the string is nonempty
<code>s.islower()</code>	there is at least one letter and all letters are lowercase
<code>s.isupper()</code>	there is at least one letter and all letters are uppercase
<code>s.isspace()</code>	all characters are whitespace and the string is nonempty

Examples:

```
"1729".isdigit()
```

returns True.

```
"-1729".isdigit()
```

returns False because - is not a digit.

```
"John Smith".isalpha()
```

returns False because the string contains a space.

Example – Exploring a Substring

```
theString = input("Enter a string: ")
```

```
theSubString = input("Enter a substring: ")
```

```
if theSubString in theString :
    print("The string contains the substring.")
    print("Count:", theString.count(theSubString))
    print("First position:", theString.find(theSubString))

    if theString.startswith(theSubString) :
        print("It appears at the beginning.")

    if theString.endswith(theSubString) :
        print("It appears at the end.")
else :
    print("The string does not contain the substring.")
```

Self Check 3.8

Question 1

How do you count blank spaces in text?

Answer

```
text.count(" ")
```

Question 2

How do you test whether a filename ends with .jpg or .jpeg?

Answer

```
filename.endswith(".jpg") or filename.endswith(".jpeg")
```

For case-insensitive checking, first normalize the name with `lower()`.

Question 3

What does `find()` return when the substring is absent?

Answer

-1.

3.9 Application: Input Validation

Input validation means checking user-supplied data before using it.

A program should not assume that every input satisfies the required rules.

Suppose an elevator accepts floors 1 through 20, except 13.

Invalid inputs include:

- 13;
- 0 or a negative number;
- a number greater than 20.

A validated version is:

```
floor = int(input("Floor: "))

if floor == 13 :
    print("Error: There is no thirteenth floor.")
elif floor <= 0 or floor > 20 :
    print("Error: The floor must be between 1 and 20.")
else :
    actualFloor = floor

    if floor > 13 :
        actualFloor = floor - 1

    print("The elevator will travel to the actual floor", actualFloor)
```

The key pattern is:

Read input

↓

Validate input

↓

Invalid? → report error

Valid? → process data

3.9.1 Range Validation

Example:

```
if age < 0 or age > 130 :
    print("Invalid age")
else :
    # Safe to process the value.
    ...
```

3.9.2 Validating Choice Inputs

Suppose the user must enter s or m.

One approach is:

```
status = input("Enter s or m: ")

if status == "s" or status == "S" :
    ...
elif status == "m" or status == "M" :
    ...
else :
    print("Invalid status")
```

A more scalable technique is to normalize first:

```
status = input("Enter s or m: ")
status = status.lower()

if status == "s" :
    ...
elif status == "m" :
    ...
else :
    print("Invalid status")
```

Similarly, multi-character codes can be normalized:

```
country = input("Country code: ")
country = country.upper()
```

Then the program only needs to compare against one representation.

3.9.3 A Limitation at This Stage

This code:

```
floor = int(input("Floor: "))
```

will still raise an exception if the user types something such as:

```
five
```

Checking conversion failures requires exception handling, which is introduced later in the textbook.

At this stage, distinguish between:

- **validating the value after conversion** — for example, checking whether it lies in a range;
 - **handling a failed conversion** — a later topic involving exceptions.
-

Special Topic 3.6 – Terminating a Program

For small text-based programs, invalid input can sometimes be handled by terminating immediately.

```
from sys import exit

response = input("Enter y or n: ")

if not (response == "y" or response == "n") :
    exit("Error: you must enter y or n.")
```

After validation succeeds, later code can assume the input satisfies the required rule.

Use early termination carefully. In larger programs, functions and exceptions often provide better structure, but those ideas are introduced later.

Special Topic 3.7 – Interactive Graphical Programs

Decision logic applies to graphical programs as well.

A graphical program may:

1. read input;
2. validate the coordinates or dimensions;
3. draw only if the values are valid;
4. react to a mouse click and branch according to its location.

The key lesson is that graphical input does not eliminate the need for validation.

Computing & Society 3.2 – Artificial Intelligence

The chapter introduces artificial intelligence as a broader context for decision-making software. Traditional programs follow explicitly designed rules, while AI research aims to build systems capable of behavior that appears more flexible or intelligent.

For this lesson, the connection is conceptual: even sophisticated systems must ultimately represent information, evaluate alternatives, and select actions. Later AI techniques may learn parts of those decision rules from data, but understanding explicit control flow remains foundational programming knowledge.

Worked Example 3.2 – Intersecting Circles

The chapter's graphical worked example determines how two circles relate to one another.

Suppose the circles have centers:

```
(x0, y0)
(x1, y1)
```

and radii:

```
r0  
r1
```

The distance between the centers is:

```
from math import sqrt
```

```
dist = sqrt((x1 - x0) ** 2 + (y1 - y0) ** 2)
```

The program can classify the relationship using decisions:

```
if dist > r0 + r1 :  
    message = "The circles are separate."  
elif dist < abs(r0 - r1) :  
    message = "One circle is inside the other."  
elif dist == r0 + r1 :  
    message = "The circles touch externally."  
elif dist == 0 and r0 == r1 :  
    message = "The circles coincide."  
else :  
    message = "The circles intersect at two points."
```

This example combines:

- input validation;
- arithmetic;
- relational operators;
- Boolean operators;
- multiple alternatives;
- geometry;
- graphical output.

For floating-point geometric computations, an approximate-equality test may be preferable when values are not exact integers.

TOOLBOX 3.2 – Plotting Simple Graphs

The chapter introduces `matplotlib.pyplot` as an optional example of using a Python library to visualize data.

A simple bar chart:

```
from matplotlib import pyplot  
  
pyplot.bar(1, 1.1)  
pyplot.bar(2, 10.0)  
pyplot.bar(3, 25.4)  
pyplot.bar(4, 44.5)  
pyplot.bar(5, 61.0)  
  
pyplot.xlabel("Month")  
pyplot.ylabel("Temperature")  
pyplot.show()
```

A simple line graph:

```
from matplotlib import pyplot

months = [1, 2, 3, 4, 5]
temperatures = [1.1, 10.0, 25.4, 44.5, 61.0]

pyplot.plot(months, temperatures)
pyplot.xlabel("Month")
pyplot.ylabel("Temperature")
pyplot.show()
```

The plotting toolbox is optional. The main chapter objective remains decision-making and Boolean logic.

Chapter 3 Summary

if Statements

- An if statement changes the path of execution according to a condition.
- An else branch provides an alternative when the condition is false.
- elif supports multiple mutually exclusive alternatives.
- A compound statement has a header and an indented block.
- Consistent indentation is part of Python syntax.
- Avoid duplicated code across branches when common work can be moved outside.

Relational Operators

< <= > >= == !=

Remember:

= assignment
== equality test

Strings can also be compared, and their comparisons are case-sensitive.

Computed floating-point values should often be compared using a tolerance rather than exact equality.

Nested Branches

- An if statement can occur inside another branch.
- Nesting is useful for decisions with several levels.
- Indentation shows which inner decision belongs to which outer branch.
- Hand-tracing helps verify nested logic.

Multiple Alternatives

Use:

```
if ... :
    ...
elif ... :
    ...
else :
    ...
```

Only the first true branch executes.

For overlapping thresholds, order tests carefully.

Flowcharts

Flowcharts provide a visual representation of:

- tasks;
- input/output;
- decisions;
- branches and joins.

Use structured branches and avoid arrows that jump into another branch.

Test Cases

Good tests include:

- at least one test for every branch;
- boundary values;
- nearby values around boundaries;
- invalid values when validation is required.

Designing expected results before coding strengthens both the algorithm and the test process.

Boolean Logic

True
False
and
or
not

- and requires all component conditions to be true.
- or requires at least one component condition to be true.
- not reverses a Boolean value.
- Python uses short-circuit evaluation.
- De Morgan's laws help simplify negated compound conditions.

String Analysis

Useful operations include:

```
substring in text
substring not in text
text.count(...)
text.find(...)
text.startswith(...)
text.endswith(...)
text.isalpha()
text.isdigit()
text.isalnum()
text.islower()
text.isupper()
text.isspace()
```

Input Validation

Before processing user input:

Read

- normalize when appropriate
- validate
- reject invalid values
- process valid data

Range checks and valid-choice checks can be implemented with conditions. Failed numeric conversions require exception handling, which is covered later.

Summary Quiz

Question 1

Which operator tests equality?

- A. =
- B. ==
- C. !=
- D. <=

Answer

B. ==

Question 2

What happens when the condition of an if/else statement is true?

- A. Both branches execute.
- B. Only the else branch executes.
- C. The if branch executes and the else branch is skipped.
- D. The program always terminates.

Answer

C

Question 3

What is the value of:

5 >= 5

- A. True
- B. False
- C. 5
- D. Error

Answer

A. True

Question 4

Which condition tests whether x lies strictly between 0 and 10?

- A. x > 0 or x < 10
- B. x > 0 and x < 10

- C. `x >= 0 or x <= 10`
- D. `not x`

Answer

B

Question 5

What is the result of:

`True or False`

Answer

True.

Question 6

Why is this test risky after floating-point calculations?

`if result == expected :`

Answer

Because tiny roundoff errors may make two mathematically equal values differ in their floating-point representations. A tolerance comparison may be more appropriate.

Question 7

Which structure is usually best for mutually exclusive grade categories?

- A. Several independent `if` statements
- B. `if/elif/else`
- C. Only an `else` statement
- D. No conditions

Answer

B. `if/elif/else`

Question 8

What does this expression test?

`".csv" in filename`

Answer

Whether the exact substring `".csv"` occurs anywhere in `filename`.

Question 9

What does `text.find("abc")` return if `"abc"` is absent?

- A. False
- B. None
- C. -1
- D. Error

Answer

C. -1

Question 10

What is a boundary test for:

```
if score >= 50 :
```

Answer

50 is the key boundary. 49 and 51 are also useful neighboring tests.

Question 11

Why can the order of conditions matter in an `if/elif` chain?

Answer

Because Python stops at the first true branch. A broad condition placed too early can prevent a more specific later condition from ever being reached.

Question 12

What is the main purpose of input validation?

- A. Make the code longer.
- B. Ensure user-supplied values satisfy the program's assumptions before they are processed.
- C. Convert all values to strings.
- D. Remove all `if` statements.

Answer

B

Review Exercises

Exercise 1. Trace a Two-Way Decision

Determine the final value of `fee`:

```
age = 16

if age < 18 :
    fee = 5
else :
    fee = 10
```

Answer

fee is 5.

Exercise 2. Choose the Correct Relational Operator

Complete the condition so that the message is printed when temperature is at most 0.

```
if temperature ____ 0 :
    print("Freezing or below")
```

Answer

`<=`

Exercise 3. Rewrite Without Duplication

Improve:

```
if quantity >= 10 :
    rate = 0.90
    total = rate * price * quantity
else :
    rate = 1.00
    total = rate * price * quantity
```

Sample Answer

```
if quantity >= 10 :
    rate = 0.90
else :
    rate = 1.00

total = rate * price * quantity
```

Exercise 4. Test Cases

For:

```
if balance < 0 :
    status = "Overdrawn"
else :
    status = "OK"
```

propose three useful test values.

Sample Answer

```
-1  → Overdrawn
0   → OK        (boundary)
1   → OK
```

Exercise 5. Boolean Expressions

Write conditions for:

1. x and y are both positive.
2. At least one of x and y is negative.
3. age is between 18 and 65, inclusive.
4. answer is neither "y" nor "n".

Sample Answer

```
x > 0 and y > 0
x < 0 or y < 0
age >= 18 and age <= 65
answer != "y" and answer != "n"
```

Exercise 6. Analyze a String

Given:

```
filename = "report.Final.PDF"
```

write a case-insensitive test for whether the filename ends in .pdf.

Answer

```
filename.lower().endswith(".pdf")
```

Practice Exercises

Exercise 1. Pass / Fail / Invalid Score

Read an exam score.

- If the score is below 0 or above 100, print an error.
- Otherwise, print "Pass" for scores of at least 50 and "Fail" for lower scores.

Draw a flowchart before writing Python.

Exercise 2. Largest of Two Numbers

Read two numbers and print:

- the larger value;
- "Equal" if they are the same.

Use an if/elif/else statement.

Exercise 3. Largest of Three Numbers

Read three numbers and determine the largest.

First solve the task using nested decisions. Then consider whether the structure can be made clearer with Boolean expressions or multiple alternatives.

Exercise 4. Shipping Rule

Read:

- destination country;
- state or province.

Normalize both inputs with upper().

Use the following rules:

- continental USA: \$5;
- Alaska or Hawaii: \$10;
- outside the USA: \$15.

Print the shipping charge with two decimal places.

Exercise 5. Grade Classification

Read a score from 0 to 100 and assign:

90–100 → A
80–89 → B
70–79 → C
60–69 → D
0–59 → F

Reject invalid scores before classifying them.

Create test cases for:

-1, 0, 59, 60, 69, 70, 79, 80, 89, 90, 100, 101

Exercise 6. Username Validation

Read a username and check that:

- it is not empty;
- it contains only letters and digits;
- it contains at least one character.

Use string-analysis methods from Section 3.8.

Exercise 7. File-Type Classifier

Read a filename and classify it as:

- Python source: .py;
- notebook: .ipynb;
- CSV data: .csv;
- text file: .txt;
- other.

The check should be case-insensitive.

Exercise 8. Approximate Equality

Read two floating-point numbers and determine whether they differ by less than 0.001.

Do not use exact equality for the approximate comparison.

Exercise 9. Triangle Type from Side Lengths

Read three positive side lengths.

First validate that they can form a triangle:

```
a + b > c
and
b + c > a
and
c + a > b
```

If valid, classify the triangle as:

- equilateral;
- isosceles;
- scalene.

Prepare branch and boundary tests before coding.

Exercise 10. Simple Login-Code Check

Read a short access code.

Validation rules:

- exactly six characters;
- only letters and digits;
- must contain the substring "NEU" ignoring case.

Print a specific message for valid or invalid input.

Open Exercises

Exercise 1

Give three real-world examples where a decision has exactly two outcomes. Express each rule in pseudocode.

Exercise 2

Give a real-world example that requires at least four mutually exclusive alternatives. Explain why an if/elif/else structure is appropriate.

Exercise 3

Choose one condition involving a range and identify:

- a typical valid value;
- a lower-boundary value;
- an upper-boundary value;
- a value just below the range;
- a value just above the range.

Exercise 4

Create an example in which independent if statements are correct but an if/elif chain would be wrong. Explain why more than one action may need to occur.

Exercise 5

Write a complicated Boolean condition involving not, and, and or. Then simplify it using De Morgan's laws.

Key Terms

Term	Meaning
Decision	Choosing an action according to a condition
Condition	Expression evaluated as true or false
if statement	Executes a block when a condition is true
else branch	Alternative block when an if condition is false
elif	Adds another condition to a multi-way decision
Compound statement	Statement with a header and one or more nested statements
Statement block	Group of statements at the same indentation level
Indentation	Leading spaces that define block structure in Python
Relational operator	Operator that compares two values
Equality test	Comparison using ==
Boundary value	Value at the edge of a decision range
Floating-point tolerance	Small acceptable difference used for approximate comparison
Nested branch	Decision statement inside another branch
Hand-tracing	Manually simulating program execution
Multiple alternatives	Decision with more than two possible branches
Flowchart	Diagram showing tasks and control flow
Branch coverage	Testing each possible branch of a decision
Test case	Input plus expected behavior used to verify a program
Boolean	Data type with values True and False
Flag	Boolean variable representing a condition or state
Boolean operator	and, or, or not
Short-circuit evaluation	Stopping Boolean evaluation once the result is known
De Morgan's laws	Rules for transforming negated and/or expressions
Substring	String contained within another string
Membership test	Test using in or not in
Input validation	Checking that input satisfies required constraints
Normalization	Converting input to a consistent form before comparison

Study Guidance

After Lesson 3, students should no longer think of a program as only a fixed list of calculations. A program can now **choose its path**.

A useful workflow for decision problems is:

- Understand the rule
- identify the possible outcomes
- formulate the condition(s)
- check boundary values

- draw a flowchart when useful
- write pseudocode
- choose if / if-else / if-elif-else / nested if
- remove duplicated work
- prepare test cases for every branch
- implement in Python
- run ordinary, boundary, and invalid-input tests
- trace unexpected behavior

A useful practice cycle is:

Predict → Trace → Run → Compare → Explain → Modify → Test boundaries → Practice

The most important habit from this chapter is to treat every decision as something that must be **designed and tested**, not merely typed.